

POSIX over object storage — Databricks and flint

How Databricks mounts buckets for /Volumes, whether it uses a sidecar or a CSI node driver, where its compute runs — Kubernetes and on-premises included — and how the whole design compares with **flint passthrough** (FUSE) and **flint lean** (a synced local tree).

Based on public sources only. Everything this document says about Databricks was gleaned from public sources — Databricks documentation, knowledge-base articles, engineering and product blogs, the SIGMOD 2025 Lakeguard paper, a KubeCon EU 2023 talk, community forum threads and third-party write-ups — read on 2026-09-16. Databricks' implementation is closed: nothing here comes from Databricks source code, from a Databricks employee in private, or from a test on a Databricks workspace. Each Databricks claim is graded DOC SEEN or INFER, and page 9 lists the source behind every one. Where Databricks has not said how something works, this document says so rather than guessing silently.

The question. Databricks gives its users POSIX paths over object storage through FUSE — /Volumes for Unity Catalog volumes, /Workspace for workspace files, /dbfs for the legacy DBFS — and that covers much of what flint's front ends exist to provide. How does it mount the buckets? Is the daemon a sidecar, a CSI node driver, or something else? Does Databricks run on Kubernetes, and does any of it run on-premises? What carries over to flint, and what does not?

The grades. DOC stated by Databricks — documentation, KB, blog, paper, talk, or a Databricks employee in public. SEEN observed or reported by a third party — a forum answer, a blog's command output, a community project. INFER inferred in this document from documented behaviour and stated nowhere. flint's claims carry no grade: their source of record is this repository — `spdk-csi-driver/src/s3csi/`, `crates/flint-s3-worker/`, `lean/`, `docs/flint-lean-for-agent-fleets.md`, `docs/plans/csi-node-mount-design.md` and the passthrough, lean and ACID posters beside this directory.

Regenerating. `docs/architecture/databricks/databricks-visio.py` draws the seven figures as one Visio file (`flint-vs-databricks.vsd`), with the family's kit (`vsdxkit.py`, `dataflowkit.py`) and its gates; it also writes the SVGs this document references and an EMF per figure for Office. `build.sh` runs the gates, checks every reference, renders this PDF with Chrome and checks one PDF page per section. Edit the HTML or the script, never the SVGs or the PDF.

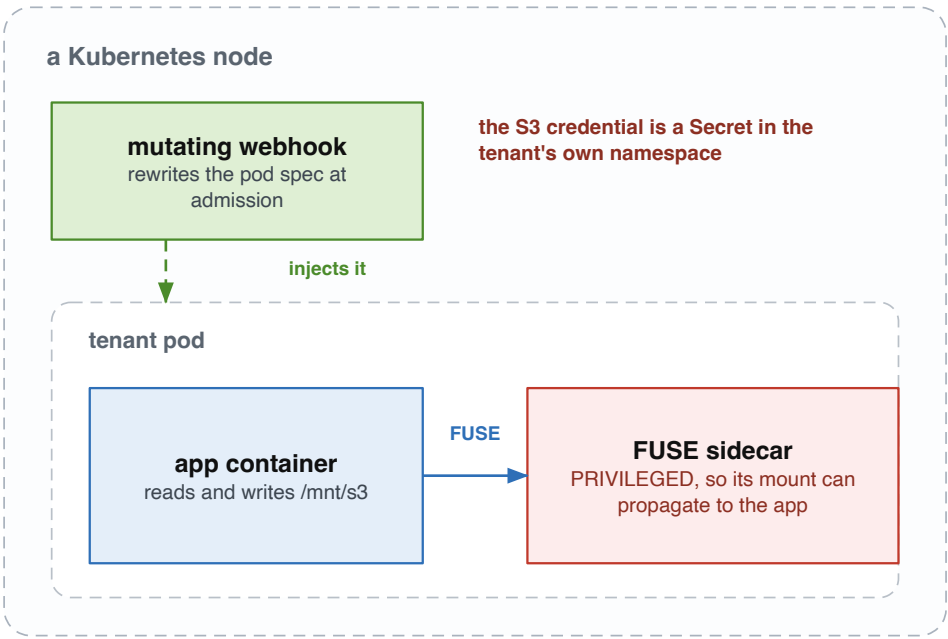
- 1 • The short answer** — FUSE is a feature of Databricks' managed service; three places a FUSE daemon can live, and which one Databricks and flint use.
- 2 • Where Databricks compute runs** — VMs on AWS and Azure, GKE on Google Cloud until 2025, Kubernetes with Kata micro-VMs for serverless, and nothing on-premises.
- 3 • A classic Databricks node** — the three FUSE trees, the Spark runtime that bypasses them, the per-user sandboxes, and the one box nobody outside can draw.
- 4 • Six doors, one contract** — a volume is a catalog object; the doors that reach it; what the FUSE door refuses, why, and the documented way round.

- 5 • flint on Kubernetes** — one CSI node plugin, a worker pod per mount, a broker for credentials; passthrough mounts FUSE, lean checks out a local tree.
- 6 • Where a write lands** — /Volumes and passthrough make the same promise; lean makes a different one, and automates Databricks' own workaround.
- 7 • Side by side** — Databricks /Volumes, flint passthrough and flint lean across seventeen rows.
- 8 • What carries over** — why the shared mount does not port to a shared Kubernetes node, what is worth borrowing, and the experiment that would settle the inferences.
- 9 • Sources** — every Databricks claim, its grade, and where it comes from.

Databricks' FUSE support is a feature of its managed service: the mounts exist only on compute Databricks provisions and runs, and there is no Databricks FUSE client for your own machines. On classic compute the daemons are started by Databricks' own runtime inside its container — neither a sidecar nor a CSI driver. Where Databricks does run Kubernetes, for serverless, the one CSI driver it has described serves local disk, not a bucket. flint does the Kubernetes-native thing: a CSI node plugin makes the mount, and a worker pod serves it.

A · a sidecar inside the tenant's pod

how flint delivered mounts until v1.45.0 — retired



WHERE THE PRIVILEGE IS
a privileged container inside every tenant pod

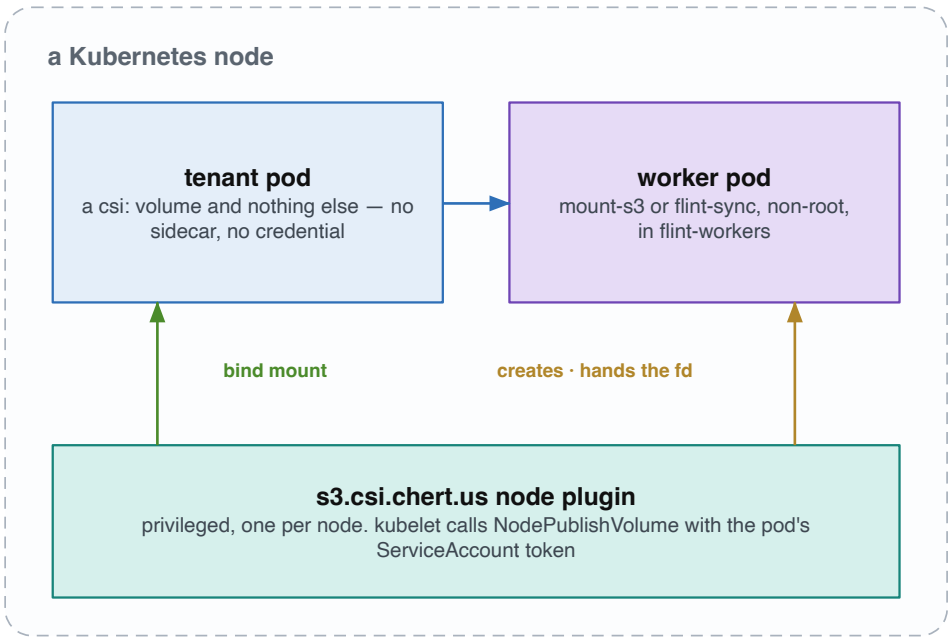
HOW MANY MOUNTS
one per pod

HOW THE CALLER IS KNOWN
as the pod — but the tenant controls the container that serves the mount

WHERE THE CREDENTIAL IS
a Secret in the tenant's namespace

B · a CSI node plugin and a worker pod

flint passthrough and flint lean today · s3.csi.chert.us



WHERE THE PRIVILEGE IS
one privileged node plugin per node; the worker that serves the mount runs non-root

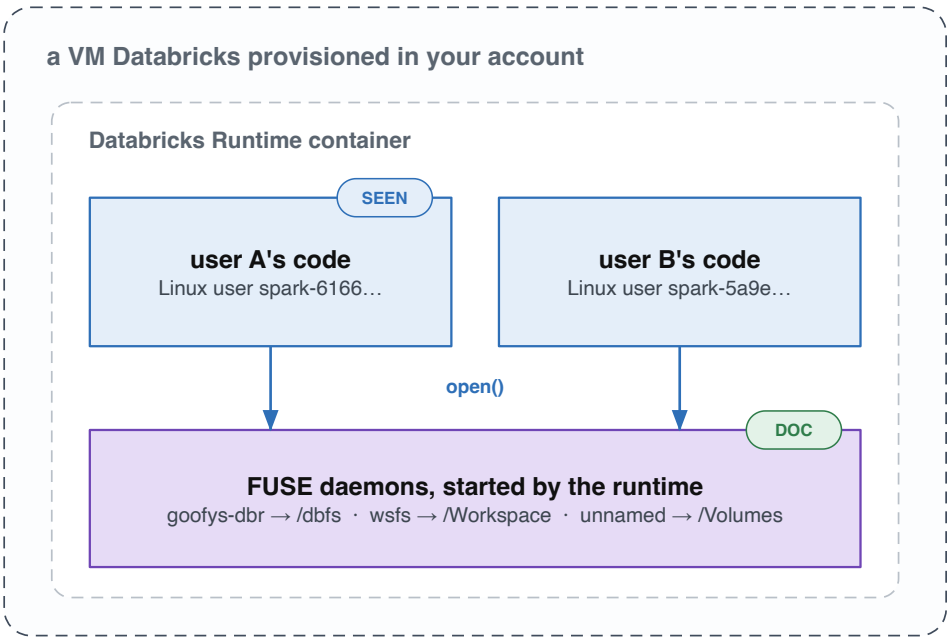
HOW MANY MOUNTS
one per pod — one worker per published volume

HOW THE CALLER IS KNOWN
the ServiceAccount kubelet asserts at NodePublishVolume, checked against spec.consumers

WHERE THE CREDENTIAL IS
short-lived keys from flint-s3-broker, to the worker only

C · a daemon the platform's runtime starts

Databricks classic compute on AWS and Azure



WHERE THE PRIVILEGE IS
the platform's own runtime, on a machine the platform provisioned

HOW MANY MOUNTS
one per filesystem per node, shared by every user and every volume on it

HOW THE CALLER IS KNOWN
a Linux user the platform created for each user; how the daemon checks it is not public

WHERE THE CREDENTIAL IS
short-lived, path-scoped credentials from Unity Catalog

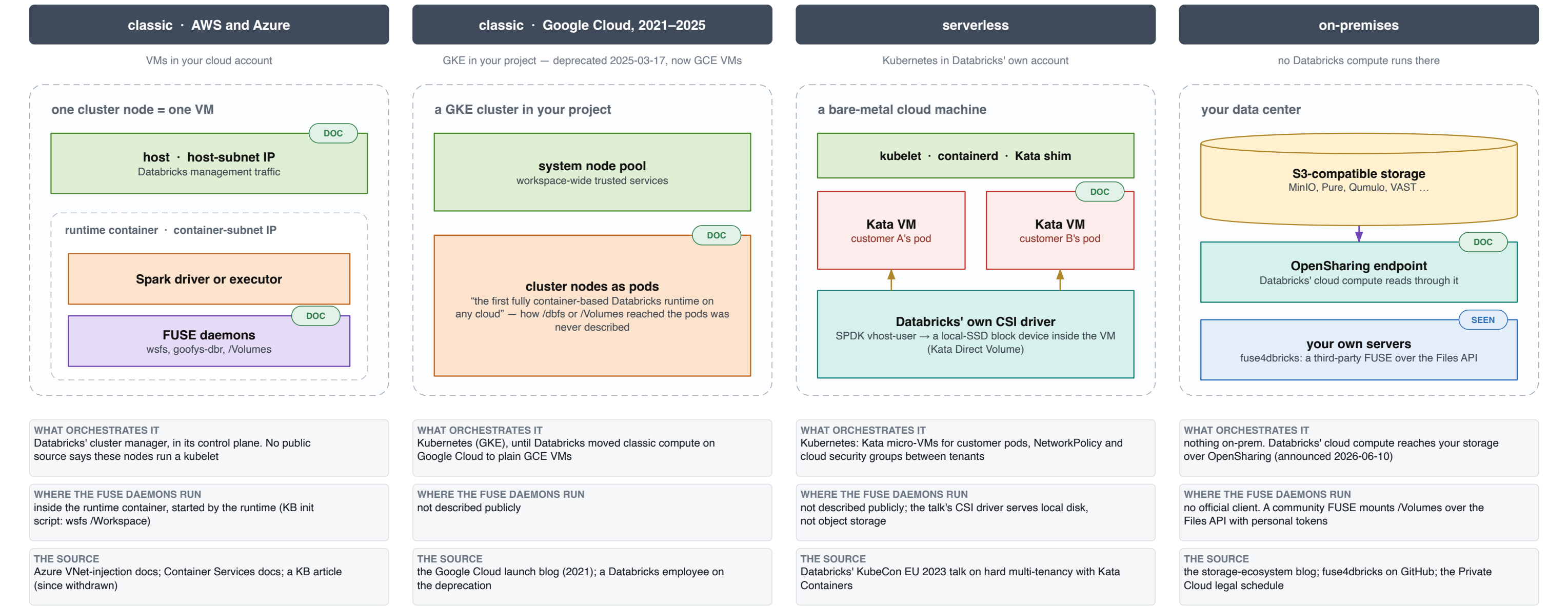
It is a SaaS feature. /Volumes, /Workspace and /dbfs exist on classic clusters — VMs Databricks creates in your cloud account and controls from its hosted control plane — and on serverless compute in Databricks' own account [DOC](#). The mounts also lean on hosted services: Unity Catalog decides who may read or write a volume and issues the short-lived storage credentials. Nothing is published to install on your own servers or Kubernetes clusters, and no Databricks compute runs on-premises (page 2). Outside Databricks compute the same files are reached through the Files API, the SDKs and the CLI, or a community FUSE client such as fuse4dbricks [SEEN](#).

Model C, not A or B. Databricks owns the whole machine, so FUSE is plumbing the runtime starts: goofys-dbr for /dbfs, wsfs for /Workspace [DOC](#), and a daemon Databricks has not named for /Volumes (page 3). One mount per filesystem serves every user on the node, which is safe only because Databricks creates the users (page 8). No public source shows a sidecar or a CSI driver serving /Volumes in any Databricks shape. Databricks' KubeCon EU 2023 talk does describe a CSI driver of its own — for serverless pods in Kata micro-VMs, handing them local SSD through SPDK vhost-user [DOC](#) — which is block storage, not a bucket.

flint is model B, and was model A until v1.45.0. A sidecar needs a privileged container in the tenant's pod (the only way to get Bidirectional mount propagation), runs an image the tenant controls, keeps the credential in the tenant's namespace, and needs a mutating webhook to inject it. The s3.csi.chert.us node plugin removes all four: it performs the one privileged act, mount(2), itself; hands the descriptor to a non-root worker pod; and leaves the tenant pod declaring nothing but a csi: volume (page 5).

“FuseMounter.” No public Databricks document, KB article, forum thread or repository found for this document names a component called FuseMounter. If the name appears in a cluster's driver log it is most likely an internal class, and the experiment on page 8 would place it.

Databricks runs on Kubernetes in some shapes and not in others, and none of it runs on-premises. Classic compute on AWS and Azure is documented as VMs with a host address and a container address. Classic compute on Google Cloud ran on GKE from its 2021 launch until GKE-based compute was deprecated on 2025-03-17 in favour of GCE VMs. Serverless compute runs on Kubernetes in Databricks' own account, each customer pod in a Kata Containers micro-VM. Databricks' own services run on Kubernetes on all three clouds.



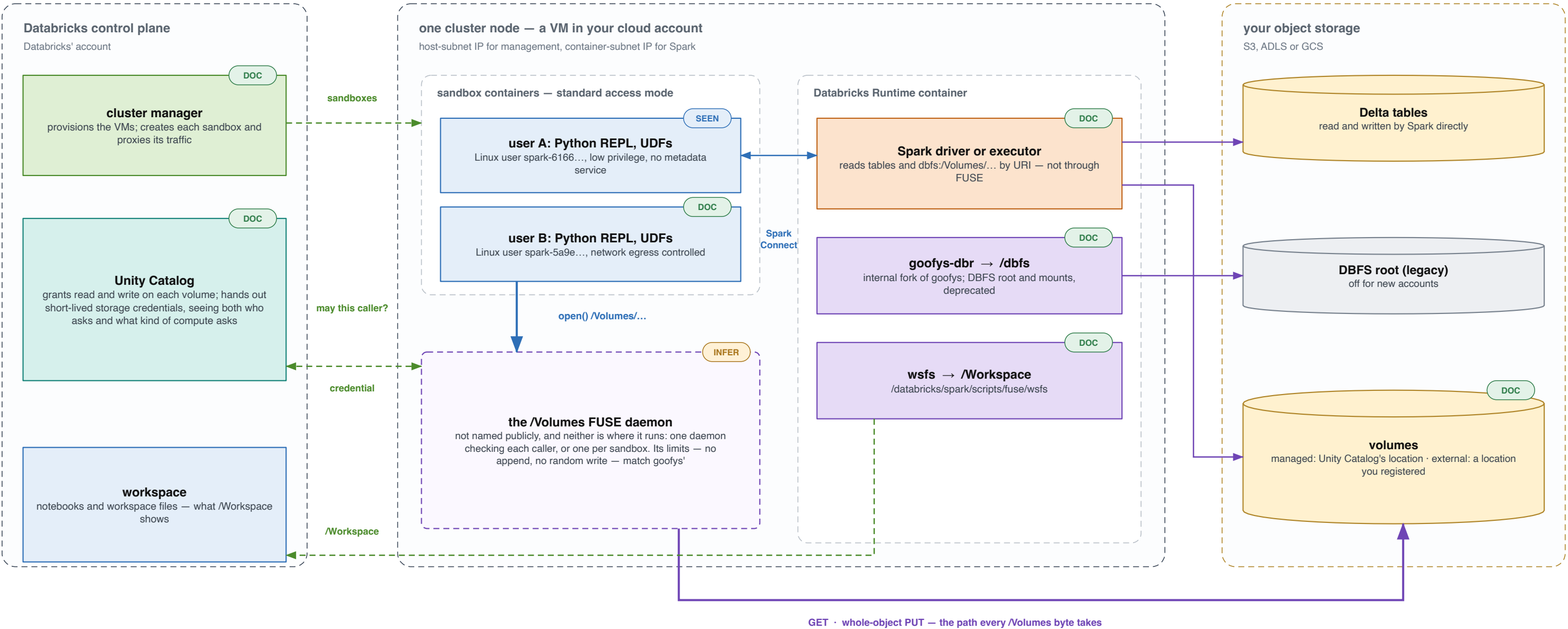
Classic, AWS and Azure. Every cluster node needs two addresses, “one IP address for the host in the host subnet and one IP address for the container in the container subnet”, and Spark's driver and executors take the container addresses [DOC](#). With Container Services the launch order is documented: acquire VMs, pull the image, create a container, copy the runtime in, run init scripts — Docker's `CMD` and `ENTRYPOINT` are ignored and the container must start as root [DOC](#). No public source says these VMs run a kubelet; asked whether Azure Databricks can use Kubernetes instead of VMs, a Databricks MVP answered “it's VM's and you can't choose” [SEEN](#), which describes what customers see rather than what the host runs.

Classic, Google Cloud. The 2021 launch was “built on Google Kubernetes Engine (GKE) ... the first fully container-based Databricks runtime on any cloud” [DOC](#), with a system node pool for workspace-wide trusted services [DOC](#). A Databricks employee confirmed on the community forum that GKE-based compute was deprecated as of 2025-03-17 and that classic compute on Google Cloud is moving to a pure GCE VM architecture [DOC](#). How `/dbfs` or `/Volumes` reached those pods was never described.

Serverless. The KubeCon EU 2023 talk (Shuo Chen, Databricks) describes Kubernetes clusters in Databricks' account running untrusted customer pods beside trusted Databricks services: each customer pod in a Kata micro-VM with dedicated cores, memory and disk partitions on a bare-metal cloud machine, tenants separated by NetworkPolicy and cloud security groups [DOC](#). Vanilla volume support through virtio-fs was too slow for Spark's scratch disk, so Databricks wrote its own CSI driver that passes an SPDK vhost-user block device into the VM (Kata Direct Volume) — the same SPDK-and-CSI pairing flint's block driver is built on. The talk says nothing about `/Volumes`. A host mount reaches a Kata guest only through the shared filesystem the talk found slow, so the `/Volumes` daemon plausibly runs inside the VM [INFER](#). Separately, Databricks' engineering blog says it orchestrates its services with Kubernetes on AWS, Azure and GCP, with OS images that bootstrap cloud VMs into Kubernetes nodes [DOC](#).

On-premises. Databricks sells no on-premises deployment [SEEN](#). Its June 2026 storage-ecosystem announcement brings on-premises data to Databricks' cloud compute — a storage partner (MinIO generally available; Pure, Qumulo and VAST in preview) exposes an OpenSharing endpoint that is registered in Unity Catalog — rather than bringing compute or mounts on-premises [DOC](#). A “Private Cloud” legal schedule describes platform services “fully deployed into a Customer Cloud Environment” and operated by the customer [DOC](#); it has no public technical documentation.

One classic node carries three FUSE trees, a Spark runtime that uses none of them, and — on standard access mode — a sandbox per user. Unity Catalog, in the control plane, is where a volume's permissions and credentials come from. The one box on the page nobody outside Databricks can draw with confidence is the one this document is about: the `/Volumes` daemon, drawn dashed.



Three trees, two named. `/dbfs` is served by `goofys-dbr`, Databricks' internal fork of the open-source `goofys` S3 filesystem, extended for Azure storage (DOC); the KB calls the mode FUSE v2, available since DBR 6.0. `/Workspace` is served by `wsfs`, started as `/databricks/spark/scripts/fuse/wsfs` `/Workspace` with its log in `/databricks/data/logs/wsfs.log` (DOC) (the KB article has since been withdrawn). `/Volumes` is served by a daemon no public source names. Its documented limits — no direct append, no random write, no sparse files — are exactly FUSE v2's, which is the whole basis for inferring `goofys` lineage (INFER).

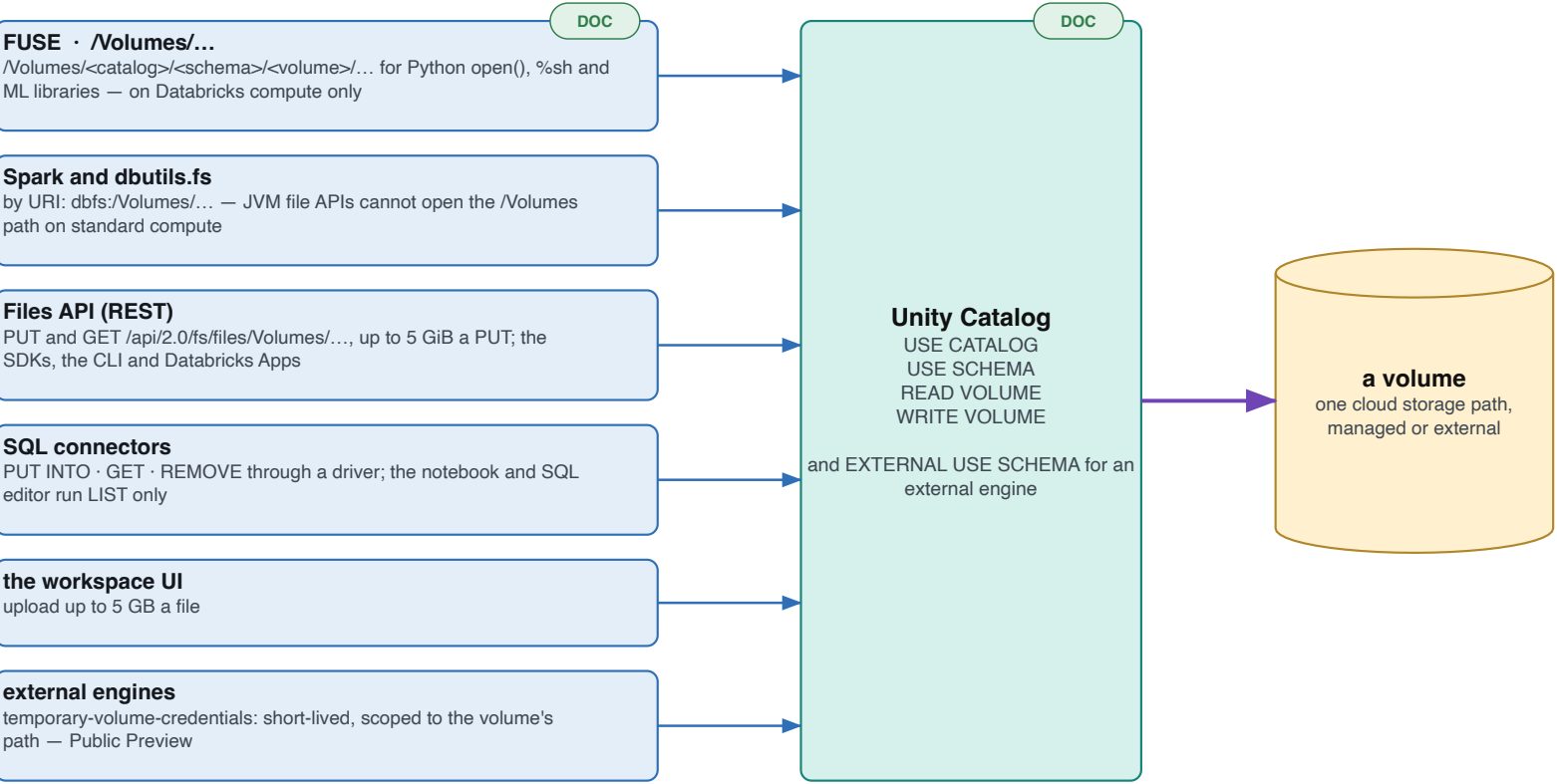
Spark does not use FUSE. Tables, and volume files read by Spark, are addressed by URI (`dbfs:/Volumes/...`, `s3://...`) and go from the JVM to storage directly (DOC). On standard compute JVM file APIs cannot open `/Volumes` paths at all, and Scala gets FUSE only on dedicated compute from DBR 14.3 — except in subprocesses it starts (DOC). FUSE exists for the Python libraries and shell tools that insist on a local path.

The users are the platform's. On standard access mode, user code — the Python REPL and UDFs — runs in sandbox containers the cluster manager creates and proxies, with egress controlled through network namespaces, as a low-privilege user that cannot reach the instance metadata service (DOC). A 2022 blog showed two users' notebooks running as distinct Linux users, `spark-6166cfd7-...` and `spark-5a9eefa7-...` (SEEN). DBFS root and DBFS mounts get no FUSE on this compute at all (DOC): a DBFS mount carried one credential for the whole cluster, which cannot be checked per user.

What is inferred. Where the `/Volumes` daemon runs, and how it tells users apart. Either one daemon per node authorises each request by the caller's uid — every FUSE request carries the caller's uid, gid and pid — or each sandbox gets its own daemon carrying its user's identity (INFER). Both fit the public record. Either way the answer comes from Unity Catalog, which the Lakeguard paper says identifies “the individual identity and compute type” behind every storage-credential request (DOC).

FUSE is one of six ways into a volume, and every one of them is authorised by Unity Catalog grants on the volume. The FUSE door is the narrowest: it writes whole files front to back and refuses the rest — and Databricks documents the refusals, the reason for them, and the way round.

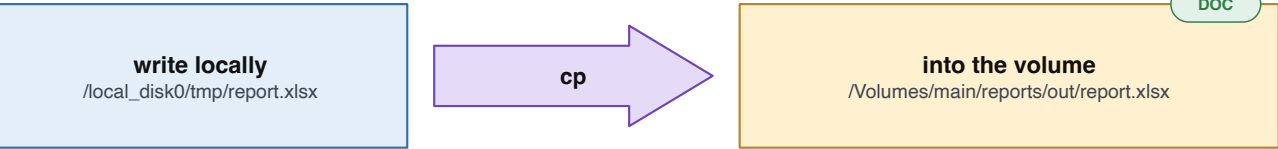
six doors to one volume — every one of them asks Unity Catalog



what the FUSE door refuses

write sequentially, then close()	the file becomes one object DOC
append to an existing file	OSError errno 95, Operation not supported
random write — seek, then write	not supported: zip and Excel writers fail
a sparse file	not supported: copy with cp --sparse=never
open /Volumes from JVM file APIs on standard compute	not supported; Scala gets FUSE on dedicated compute, DBR 14.3 and above
/dbfs FUSE on standard compute	DBFS root and mounts have no FUSE there

the documented way round, for zip, Excel and appends



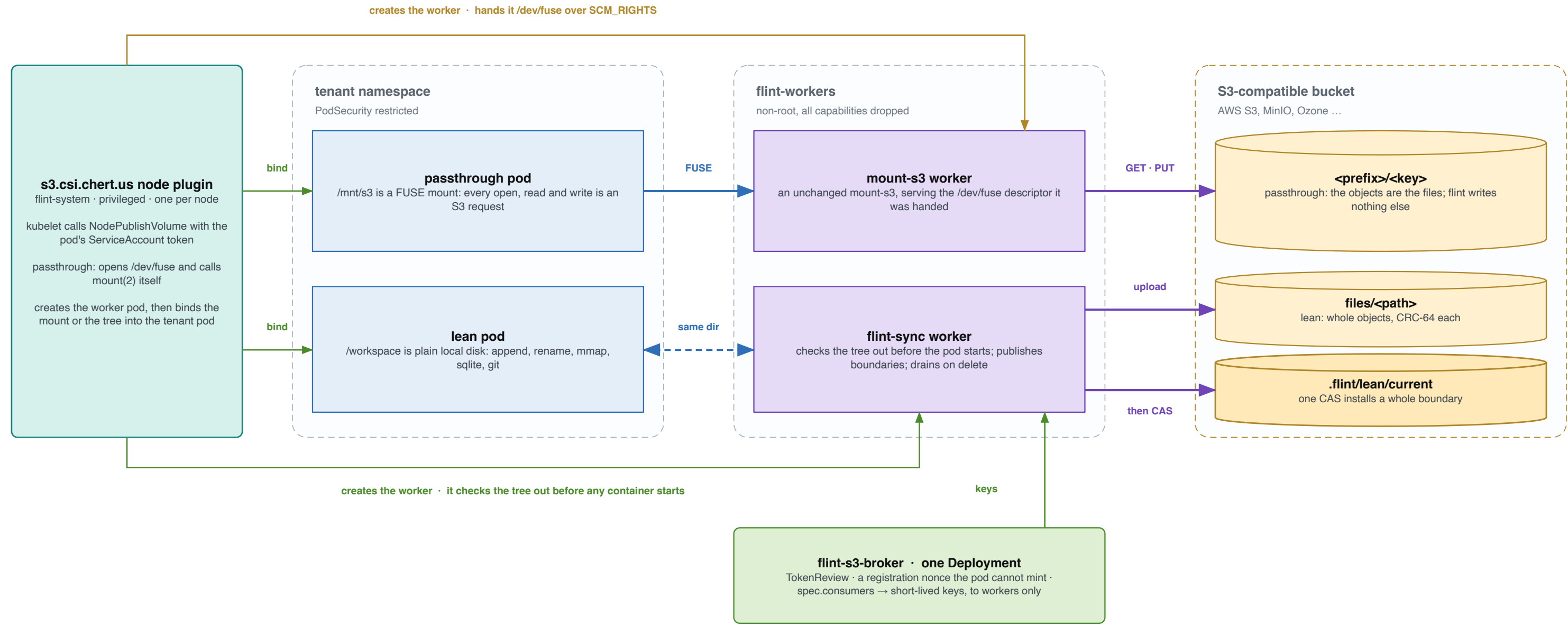
and for appends that must survive across sessions, a KB article says to ask the account team for an NFS mount

A volume is a catalog object, not a mount. A volume is a governed path — managed, in Unity Catalog's storage location, or external, in a location the customer registered — and `/Volumes/<catalog>/<schema>/<volume>` is the catalog's own namespace **DOC**. So one mount on a node covers every volume a user can see, no mount is made per volume, and no credential lives in any mount's configuration. A user needs `USE CATALOG`, `USE SCHEMA` and `READ VOLUME`, plus `WRITE VOLUME` to write; an external engine also needs `EXTERNAL USE SCHEMA` and receives a credential scoped to the volume's path from `temporary-volume-credentials`, in Public Preview **DOC**.

The other doors reach beyond Databricks compute. The Files API takes PUT and GET on `/api/2.0/fs/files/Volumes/...`, up to 5 GiB a request, and is what the SDKs and the CLI use **DOC**. A Databricks App is granted a volume as a resource, and its service principal receives `USE CATALOG`, `USE SCHEMA` and `READ` or `WRITE VOLUME` **DOC**; a community thread reports that writing `/dbfs/Volumes` from inside an App fails with Errno 5 **SEEN**. SQL connectors can PUT, GET and REMOVE, while the notebook and SQL editor run LIST only; the UI uploads files up to 5 GB **DOC**.

The contract is small on purpose, and written down. Sequential write then close is supported; append fails with errno 95, random writes are not supported (so zip and Excel writers fail), and sparse files are not supported **DOC**. The KB gives the reason: the underlying storage has no append, so honouring one would mean downloading the object, changing it and uploading it again, and on a mount shared across nodes concurrent appends could corrupt the file. The documented way round is to write under `/local_disk0` and copy the result into the volume; for appends that must outlive a session, the KB says to ask the account team for an NFS mount. What is worth noticing is not the limit — every S3 FUSE shares it — but that the limit, its reason and the workaround are all published.

flint cannot own the machine, so it uses the hook Kubernetes gives a storage system: a CSI node plugin that kubelet calls before a pod's containers start. Passthrough and lean share the plugin, the worker namespace and the broker, and differ in one thing — whether the pod gets a FUSE mount or a plain local directory. Both run against any S3-compatible store, in any cluster, on-premises included.



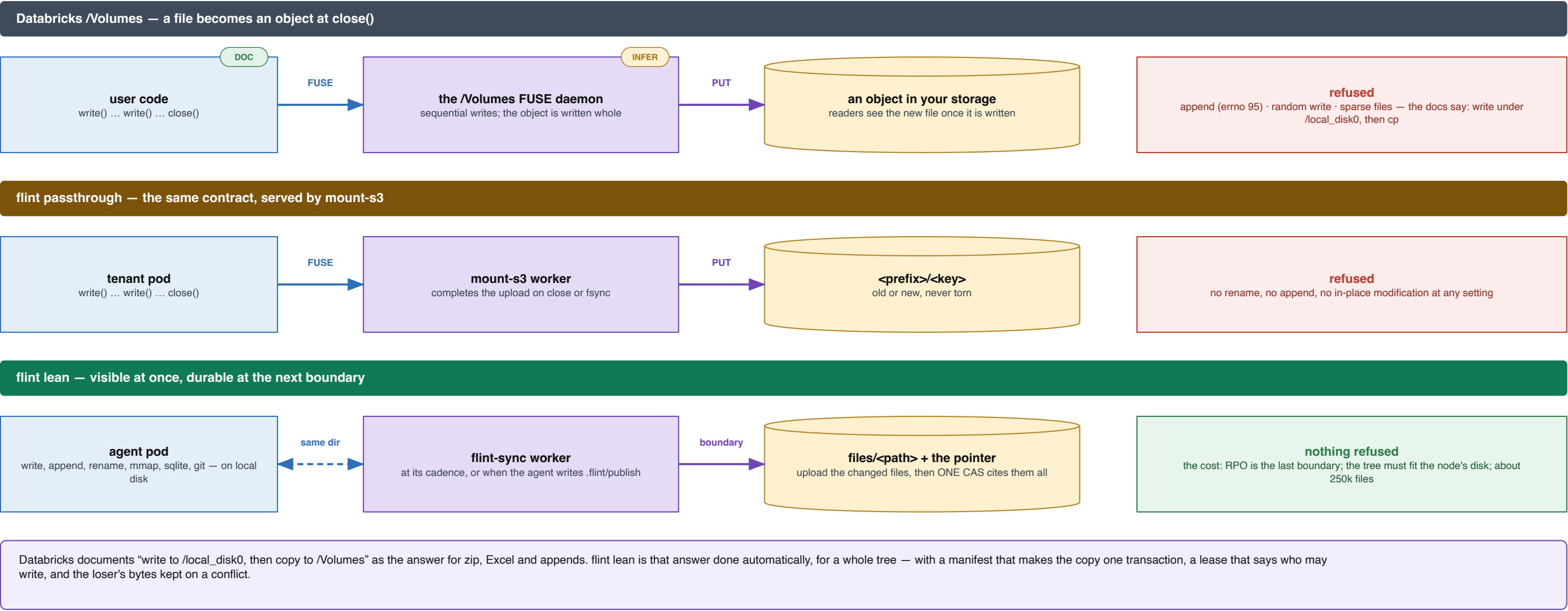
One privileged act, done by the plugin. kubelet calls `NodePublishVolume` with a `ServiceAccount` token bound to the pod. The plugin resolves the `FlintPassthroughMount` or `FlintLeanWorkspace` the pod names in its `volumeAttributes`, in the token's own namespace, and refuses unless `spec.consumers` lists the pod's `ServiceAccount` — absent means deny. For passthrough it opens `/dev/fuse` and calls `mount(2)` itself, then passes the descriptor over `SCM_RIGHTS` to a worker pod it creates in `flint-workers`, where an unchanged `mount-s3` serves it with no privilege. The result is bound into the tenant pod, which stays admissible under `PodSecurity restricted`.

Lean skips FUSE entirely. Its worker is `flint-sync`. It checks the tree out to local disk inside `NodePublishVolume`, which kubelet blocks every container on, so the agent's first line sees a complete tree. It publishes a boundary at its cadence or when the agent writes `.flint/publish` — the changed files uploaded, then one CAS on `.flint/lean/current` citing them all — and drains in `NodeUnpublishVolume`, which runs only after every container has exited. The pod sees real local POSIX, with no interception at all.

Credentials reach workers only. `flint-s3-broker`, one Deployment, verifies the token online (`TokenReview`), checks a registration nonce the pod cannot mint and `spec.consumers`, and issues short-lived keys on a loopback credentials endpoint only the worker can reach; what a given pod is actually given is decided by the broker's backend. No credential and no privileged container exists in the tenant namespace. This is flint's counterpart to Unity Catalog's credential issuing, cut to what one mount needs: an identity in, short-lived keys out.

Runs anywhere. Nothing here needs a hosted control plane: the charts install into any Kubernetes cluster, and the bucket can be AWS S3, MinIO or Apache Ozone. This is the one comparison that is not close. Databricks' mounts do not exist off Databricks' own compute; flint's exist wherever there is a Kubernetes node and an S3 endpoint, including a data center with no public cloud in it.

Databricks /Volumes and flint passthrough make the same promise, because both are FUSE over whole objects: a file becomes an object when it is closed, and anything that needs to edit an object in place is refused. flint lean makes a different promise: a write is visible at once on local disk and durable at the next boundary, and nothing is refused.

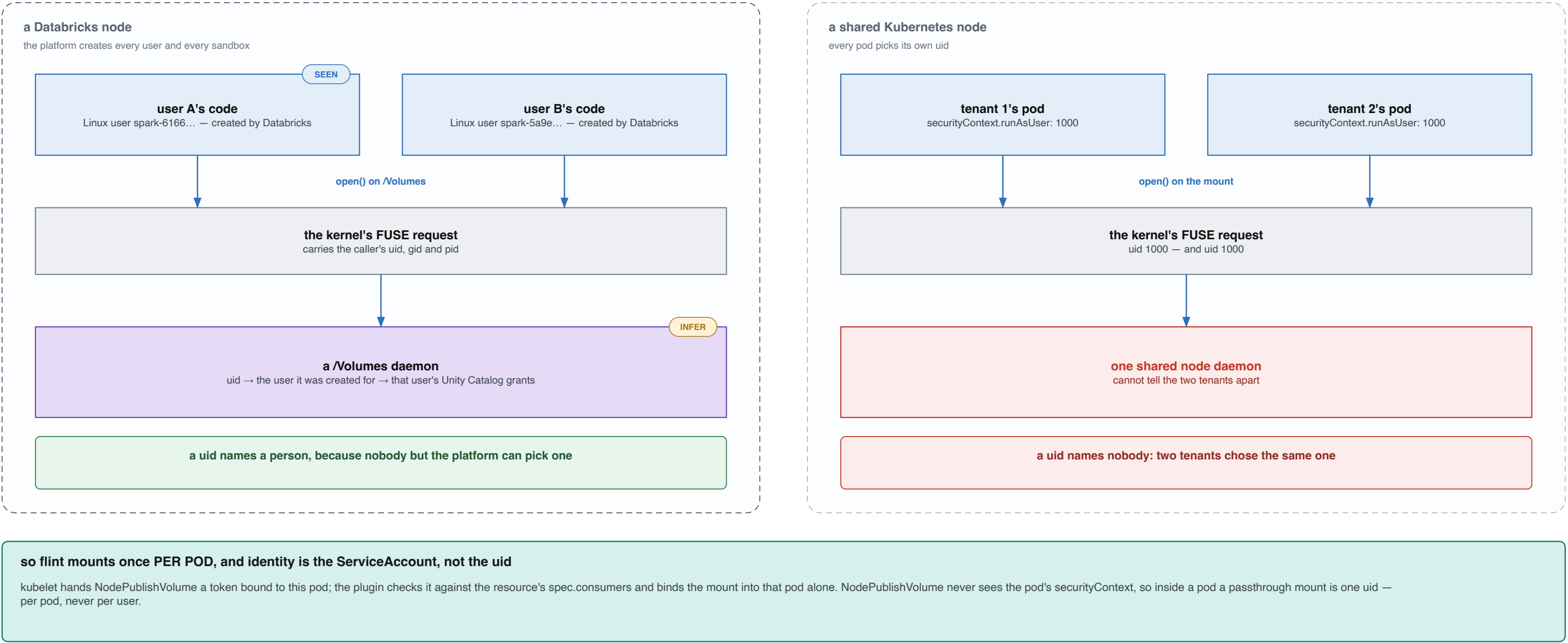


Databricks /Volumes against flint's two object-store front ends. Grades apply to the Databricks column only; flint's cells are taken from this repository.

	Databricks /Volumes	flint passthrough	flint lean
Where it runs	Databricks-operated compute only: classic VMs in your cloud account, serverless in Databricks' account DOC	any Kubernetes cluster, in a cloud or on-premises	any Kubernetes cluster, in a cloud or on-premises
Control plane	hosted by Databricks: cluster manager, Unity Catalog DOC	nothing hosted: a CRD, the node plugin and the broker, in your cluster	the same, plus a thin optional operator
Backing store	S3, ADLS or GCS; on-premises storage only as a data source for cloud compute, over OpenSharing DOC	any S3-compatible store: AWS S3, MinIO, Ozone	any S3-compatible store: AWS S3, MinIO, Ozone
How the mount arrives	started by Databricks' runtime; no public source shows a sidecar or CSI driver serving it DOC INFER	a CSI inline volume: the node plugin mounts, a worker pod serves	a CSI inline volume: the worker checks the tree out before any container starts
Privileged part	Databricks' runtime, on a VM Databricks provisioned	the node plugin, one per node	the node plugin, one per node
Mounts	one /Volumes per node for every user and volume INFER	one per pod	one per pod
Caller identity	a Linux user the platform created SEEN ; how the daemon checks it INFER	the pod's ServiceAccount, asserted by kubelet, checked against spec.consumers	the same
Authorisation	Unity Catalog grants: READ VOLUME, WRITE VOLUME DOC	spec.consumers; the broker's backend decides what a pod is given	the same, plus the workspace's claim and lease
Credential	short-lived, scoped to a path, from Unity Catalog DOC	short-lived keys from flint-s3-broker, to the worker only	the same
Data path	FUSE: each operation is an object-store request INFER	FUSE: each operation is an S3 request	local disk; S3 only at checkout and at a boundary
Refused	append (errno 95), random write, sparse files DOC	rename, append, in-place modification	nothing
Other readers see a write	when the file is closed INFER	when the file is closed	at the next boundary; at once on the pod's own disk
Atomic unit	one object	one object	one boundary: N files, one CAS
Two writers, one file	last close wins INFER	last close wins, undetected	last boundary wins, the loser's bytes kept
Size	bucket-sized, read on demand	bucket-sized, read on demand	must fit the node's disk; about 250k files in v1
Other doors to the bytes	Files API, Spark and dbutils.fs, SQL connectors, the UI, credential vending DOC	the bucket itself: any S3 client, with its own credential	flint-lean-gateway, as a REST service or a crate
Where it wins	zero setup for the user; one namespace for every volume; one authority for grants and credentials	no copy and no disk limit; plain objects other tools can read	full POSIX at local speed; a transaction per boundary

The other two front ends. flint lite (an NFS hub per share) is the answer when two pods must see one live tree, with byte-range locks — the shape Databricks reaches for when its KB tells a customer to ask for an NFS mount. flint forge (git over S3) has no Databricks counterpart.

The part of Databricks worth copying is not its FUSE daemon, which by every public sign is a goofys descendant with goofys' limits. It is everything around the mount: identity from the platform, grants and short-lived credentials from one authority, a small contract written down, and other doors to the same bytes. flint already has most of that in Kubernetes form. The one thing that does not carry over is the shared mount.



Every Databricks claim in this document, its grade, and the public source it rests on. All sources were read on 2026-09-16. Nothing here comes from Databricks source code, a private conversation, or a test on a Databricks workspace.

	claim	grade	source
1	<code>/Volumes/<catalog>/<schema>/<volume></code> is a FUSE mount giving POSIX paths to Python libraries and tools; JVM file APIs cannot use it; Scala FUSE only on dedicated compute from DBR 14.3	DOC	Work with files on Databricks
2	No direct append or random write (zip, Excel); no sparse files, copy with <code>cp --sparse=never</code> ; UI uploads up to 5 GB; the <code>/local_disk0</code> workaround	DOC	Work with files in Unity Catalog volumes
3	Append fails with errno 95; FUSE v2 (DBR 6.0+) has no append or random write; why; ask for an NFS mount for cross-session appends	DOC	KB: Operation not supported during append
4	<code>goofys-dbr</code> is Databricks' internal fork of <code>goofys</code> , adding Azure storage; <code>/databricks/spark/scripts/fuse/wsfs /Workspace</code> logging to <code>/databricks/data/logs/wsfs.log</code>	DOC	KB: Use Databricks Repos with Docker container services — withdrawn (404 on 2026-09-16); text as quoted by the search index
5	Standard compute runs user commands as a low-privilege user, cannot reach the instance metadata service, and gives DBFS root and mounts no FUSE	DOC	Standard compute limitations
6	Shared-access notebooks run as distinct Linux users, <code>spark-6166cfd7-...</code> and <code>spark-5a9eefa7-...</code> (2022)	SEEN	copdips: Databricks cluster access mode
7	User code runs in sandbox containers the cluster manager creates and proxies; egress via network namespaces; Spark reached over Spark Connect; Unity Catalog sees identity and compute type on credential requests	DOC	Lakeguard, SIGMOD 2025 · Container Services for standard compute
8	Volume credentials for external engines: <code>temporary-volume-credentials</code> , path-scoped, expiring, <code>EXTERNAL USE SCHEMA</code> , Public Preview	DOC	Unity Catalog credential vending
9	Files API: <code>/api/2.0/fs/files</code> with volume paths, up to 5 GiB per upload, ranged GET	DOC	Files API reference
10	Apps get a volume as a resource; the service principal receives <code>USE CATALOG</code> , <code>USE SCHEMA</code> , <code>READ</code> or <code>WRITE VOLUME</code>	DOC	Add a Unity Catalog volume resource to an app
11	Writing <code>/dbfs/Volumes</code> from a Databricks App fails with Errno 5	SEEN	community thread 113968
12	Two IP addresses per classic node: host subnet and container subnet	DOC	Azure Databricks VNet injection
13	Container Services launch order; <code>CMD</code> and <code>ENTRYPOINT</code> ignored; the container must start as root	DOC	Container Services · KB: custom Docker image requires root
14	Classic compute is VMs and customers cannot choose Kubernetes; SQL Serverless uses Kubernetes	SEEN	community thread 74956 (a Databricks MVP)
15	Databricks on Google Cloud “built on GKE ... the first fully container-based Databricks runtime on any cloud” (2021-02-17)	DOC	Databricks on Google Cloud launch
15b	A GKE cluster bootstrapped with a system node pool for workspace-wide trusted services	DOC	Databricks blog “How We Built Databricks on Google Kubernetes Engine (GKE)”, 2021-08-06 — the URL now redirects; text as quoted by the search index
16	GKE-based compute deprecated as of 2025-03-17; classic compute on Google Cloud moving to GCE VMs	DOC	community thread 111137 (a Databricks employee)
17	Serverless on Kubernetes in Databricks' account; customer pods in Kata micro-VMs; <code>NetworkPolicy</code> and security groups; <code>virtio-fs</code> too slow; Databricks' own CSI driver with <code>SPDK vhost-user</code> and <code>Kata Direct Volume</code>	DOC	KubeCon EU 2023, Shuo Chen (slides linked there)
18	Databricks orchestrates its services on Kubernetes clusters on AWS, Azure and GCP; OS images bootstrap cloud VMs into Kubernetes nodes	DOC	Kubernetes node upgrade using operators · Intelligent Kubernetes load balancing
19	No on-premises deployment of Databricks	SEEN	community thread 140642
20	Storage ecosystem (2026-06-10): on-premises data reached by Databricks' cloud compute over OpenSharing; MinIO GA, Pure, Qumulo, VAST in preview	DOC	Announcing the Databricks storage ecosystem
21	“Private Cloud”: platform services fully deployed into a customer cloud environment, operated by the customer	DOC	Private Cloud Schedule
22	A community FUSE client mounts <code>/Volumes</code> outside Databricks over the Files API, with personal access tokens	SEEN	zeehio/fuse4dbricks
23	The <code>/Volumes</code> daemon descends from <code>goofys</code> ; where it runs; how it tells users apart; that <code>/Volumes</code> readers see a file at close and concurrent writers are last-close-wins	INFER	from rows 1–7; settled by the notebook cell on page 8

What was searched and not found. A Databricks component named “FuseMounter”; a public description of how `/Volumes` is served on serverless compute or reached a GKE pod; any sidecar or CSI driver serving object storage in any Databricks shape. Absence from public sources is not absence from the product.

flint's side. Every flint claim comes from this repository: `spdk-csi-driver/src/s3csi/` (the node plugin, `fuse.rs`, the broker), `crates/flint-s3-worker/`, `lean/`, `docs/flint-lean-for-agent-fleets.md` (the cost table and “when not to use lean”), `docs/plans/csi-node-mount-design.md` (why the sidecar was retired), and the posters in `docs/architecture/passthrough/`, `lean/` and `acid/`.